

NAGphormer : A Tokenized Graph Transformer for node classification in large graphs

CHAE WON PARK

Network Science Lab
Dept. of Artificial Intelligence
The Catholic University of Korea

E-mail: codnjsld@catholic.ac.kr

Contents



- Introduction
 - Limitation of GNN
 - Contributions
 - Scalability
- Background
 - GNN
 - Transformer
 - Graph Transformer
- Methods
 - Hop2Token
 - NAGphormer
- Experiments
 - Comparison of Datasets
 - Ablation Study
 - Parameter Study

Limitation of GNN

- **Over-smoothing**
 - GNN이 깊어질수록 멀리 떨어진 노드들의 정보들까지 message passing 하면서, 궁극적으로 그래프 내 모든 노드 표현이 서로 비슷해져 구별하기 어려워지는 현상
 - 노드 간의 특징적인 정보가 사라져 node classification 같은 downstream task의 성능이 저하됨.
- **Over-squashing**
 - GNN의 message passing 과정에서 멀리 떨어진 노드의 정보가 고정된 크기의 벡터를 통해 전달되면서, 정보가 손실되거나 압착되어 최종 노드 표현에 충분히 반영되지 못하는 현상 (IB)
 - 멀리 떨어진 이웃의 중요한 정보가 제대로 전달되지 않아 모델이 전역적인 패턴을 학습하기 어려움.

Contributions

- **Hop2Token module**
 - 노드 주변 정보를 모은 토큰을 순서대로 나열해서 시퀀스로 만듦
- **NAGphormer model**
 - Node classification task를 위한 새로운 GT 모델
 - Hop2Token 모듈의 출력에 기반하여 미니배치 방식으로 학습하여 대규모 그래프 처리 문제를 해결함
 - Attention-based readout function을 사용하여 각기 다른 hop의 이웃들이 최종 노드 표현에 미치는 중요도를 학습하여 모델 성능을 향상시킴
- **Self-attention mechanism**
 - NAGphormer는 이 메커니즘을 통해 노드의 다중 hop 이웃 정보를 학습하고, Decoupled GCN과 같은 GNN 모델보다 표현력이 뛰어난 노드 표현을 학습할 수 있음

Scalability

- 기존 GT 모델은 복잡도가 높아서 대규모 그래프에 적용하기 어려움

$$O(N^2)$$

- NAGphormer가 이 문제점을 해결하기 위해 노드를 토큰 시퀀스로 만들어 미니배치 학습을 가능하게 함
으로써 대규모 그래프 처리가 가능함

$$O(N \cdot K^2) \approx O(N)$$

GNN

- **GCN**

- GCN layer
- 이웃 정보를 집계한 결과에 바로 특징 변환이 이루어짐

$$\mathbf{H}^{(l+1)} = \sigma(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$$

- **Decoupled GCN**

- Over-smoothing 문제 완화 (특징 변환과 이웃 집계 계산 과정을 분리)
- General form (APPNP/GPRGNN)

$$\mathbf{Z} = \sum_{k=0}^K \beta_k \mathbf{H}^{(k)}, \mathbf{H}^{(k)} = \hat{\mathbf{A}}\mathbf{H}^{(k-1)}, \mathbf{H}^{(0)} = f_{\theta}(\mathbf{X})$$

Transformer

- **Self-attention**

- Query, Key, Value

$$\mathbf{Q} = \mathbf{H}\mathbf{W}^Q, \mathbf{K} = \mathbf{H}\mathbf{W}^K, \mathbf{V} = \mathbf{H}\mathbf{W}^V$$

- Self-attention layer

$$\mathbf{H}' = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_K}} \right) \mathbf{V}$$

- Softmax로 확률 변환
- Attention score

Graph Transformer

- 그래프의 structural information을 transformer 구조로 변환하여 노드 표현을 학습시킴
 - ① 위치 인코딩을 라플라시안 고유벡터나 노드 차수에 관련된 특징 벡터로 대체 (노드의 구조적 특징 포착)
 - ② GNN을 추가적인 모듈로 사용하여 지역적 구조 정보를 더 잘 학습하게 함
 - ③ Attention score에 그래프의 구조적 정보를 bias로 통합

Hop2Token

- Multi-hop 이웃 정보 집계

- 특정 노드 v 의 k -hop 이웃의 특징을 집계 연산자를 통해 하나의 표현으로 변환함

$$\mathbf{x}_v^k = \phi(\mathcal{N}^k(v))$$

- 0-hop 이웃 = 자기 자신

$$\mathcal{N}^0(v) = \{v\}$$

- 토큰 시퀀스 생성

- 각 노드 v 에 대해, 0-hop부터 고정된 최대 hop 수 K 까지의 이웃 표현을 순서대로 나열하여 노드의 이웃 정보를 담은 시퀀스를 생성함
- 시퀀스의 각 요소 x 는 일종의 토큰으로 간주됨

$$\mathcal{S}_v = (\mathbf{x}_v^0, \mathbf{x}_v^1, \dots, \mathbf{x}_v^K)$$

Hop2Token

- 구현 방식

- 정규화된 인접 행렬과 특징 행렬을 사용하여 k-hop 이웃 특징 행렬을 얻음

$$\mathbf{X}_k = \hat{\mathbf{A}}^k \mathbf{X}$$

- 그래프의 각 노드에 대해, 해당 노드의 0-hop부터 K-hop까지의 이웃 정보를 담고 있는 특징 벡터들을 모아 놓은 3차원 텐서

$$\mathbf{X}_G \in \mathbb{R}^{n \times (K+1) \times d}$$

- 장점

- 각 노드를 토큰 시퀀스로 만들어 전체 그래프를 한 번에 처리하지 않고 노드들의 미니배치를 구성하여 학습할 수 있음. -> GPU 메모리 제약 완화, 대규모 그래프 처리 가능
- 학습 가능한 파라미터가 없어서 토큰 미리 계산 가능 -> 학습 부담 완화
- 각 hop의 이웃 정보를 별도의 토큰으로 표현함 -> 각 hop 별로 다른 중요도를 학습할 수 있음

NAGphormer

① Structural Encoding

- 노드의 속성 정보뿐만 아니라 그래프의 연결 구조 정보를 통해 더 풍부한 노드 표현을 학습함
- 속성+구조 정보 특징 행렬

$$\mathbf{X}' = \mathbf{X} \parallel \mathbf{U}$$

② Hop2Token으로 이웃 정보 시퀀스 생성

- 이전 단계에서 얻은 특징 행렬 $\mathbf{X}'$ 를 모듈의 입력으로 사용

$$\mathcal{S}_v = (\mathbf{x}_v^0, \mathbf{x}_v^1, \dots, \mathbf{x}_v^K)$$

- 각 노드는 $K+1$ 개의 토큰으로 구성된 시퀀스를 갖게 됨

③ Lineaer Projection

- 이웃 토큰 시퀀스를 transformer의 입력으로 적합한 차원으로 변환함

$$\mathbf{Z}_v^{(0)} = [\mathbf{x}_v^0 \mathbf{E}; \mathbf{x}_v^1 \mathbf{E}; \dots; \mathbf{x}_v^K \mathbf{E}]$$

NAGphormer

④ Transformer Encoder

- Multi-Head Self-Attention

$$\mathbf{Z}'_v^{(\ell)} = \text{MSA} \left(\text{LN} \left(\mathbf{Z}_v^{(\ell-1)} \right) \right) + \mathbf{Z}_v^{(\ell-1)}$$

- Position-wise Feed-Forward Network

$$\mathbf{Z}_v^{(\ell)} = \text{FFN} \left(\text{LN} \left(\mathbf{Z}'_v^{(\ell)} \right) \right) + \mathbf{Z}'_v^{(\ell)}$$

⑤ Attention-based Readout Function

$$\alpha_k = \frac{\exp((\mathbf{Z}_0 \| \mathbf{Z}_k) \mathbf{W}_a^\top)}{\sum_{i=1}^K \exp((\mathbf{Z}_0 \| \mathbf{Z}_i) \mathbf{W}_a^\top)}$$

- 최종 노드 표현

$$\mathbf{Z}_{out} = \mathbf{Z}_0 + \sum_{k=1}^K \alpha_k \mathbf{Z}_k$$

NAGphormer

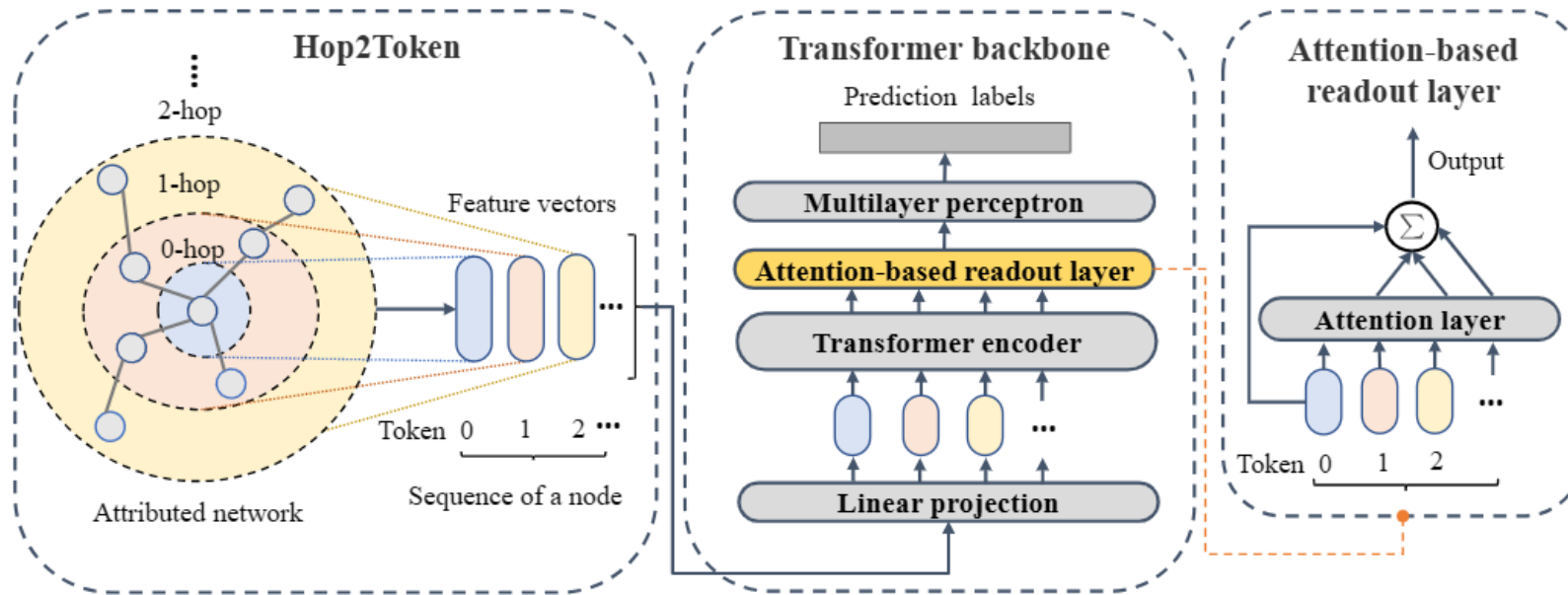


Figure 1: **Model framework of NAGphormer.** NAGphormer first uses a novel neighborhood aggregation module, Hop2Token, to construct a sequence for each node based on the tokens of different hops of neighbors. Then, NAGphormer learns the node representations using a Transformer backbone, and an attention-based readout function is developed to aggregate neighborhood information of different hops adaptively. An MLP-based module is used in the end for label prediction.

Comparison of Datasets

- 기존 GT 모델보다 NAGphormer의 성능이 높음 -> local 정보를 활용하는 것이 노드 분류에 유리함.
- 기존 GT 모델들은 그래프 내의 모든 노드를 개별적인 토큰으로 취급하여 메모리 복잡도가 올라감
- OOM(Out-of-Memory)

Table 1: Comparison of all models in terms of mean accuracy $\pm$ stdev (%) on small-scale datasets. The best results appear in **bold**. OOM indicates the out-of-memory error.

Method	Pubmed	CoraFull	Computer	Photo	CS	Physics
GCN	86.54 $\pm$ 0.12	61.76 $\pm$ 0.14	89.65 $\pm$ 0.52	92.70 $\pm$ 0.20	92.92 $\pm$ 0.12	96.18 $\pm$ 0.07
GAT	86.32 $\pm$ 0.16	64.47 $\pm$ 0.18	90.78 $\pm$ 0.13	93.87 $\pm$ 0.11	93.61 $\pm$ 0.14	96.17 $\pm$ 0.08
APNP	88.43 $\pm$ 0.15	65.16 $\pm$ 0.28	90.18 $\pm$ 0.17	94.32 $\pm$ 0.14	94.49 $\pm$ 0.07	96.54 $\pm$ 0.07
GPRGNN	89.34 $\pm$ 0.25	67.12 $\pm$ 0.31	89.32 $\pm$ 0.29	94.49 $\pm$ 0.14	95.13 $\pm$ 0.09	96.85 $\pm$ 0.08
GraphSAINT	88.96 $\pm$ 0.16	67.85 $\pm$ 0.21	90.22 $\pm$ 0.15	91.72 $\pm$ 0.13	94.41 $\pm$ 0.09	96.43 $\pm$ 0.05
PPRGo	87.38 $\pm$ 0.11	63.54 $\pm$ 0.25	88.69 $\pm$ 0.21	93.61 $\pm$ 0.12	92.52 $\pm$ 0.15	95.51 $\pm$ 0.08
GRAND+	88.64 $\pm$ 0.09	71.37 $\pm$ 0.11	88.74 $\pm$ 0.11	94.75 $\pm$ 0.12	93.92 $\pm$ 0.08	96.47 $\pm$ 0.04
GT	88.79 $\pm$ 0.12	61.05 $\pm$ 0.38	91.18 $\pm$ 0.17	94.74 $\pm$ 0.13	94.64 $\pm$ 0.13	97.05 $\pm$ 0.05
Graphormer	OOM	OOM	OOM	92.74 $\pm$ 0.14	OOM	OOM
SAN	88.22 $\pm$ 0.15	59.01 $\pm$ 0.34	89.83 $\pm$ 0.16	94.86 $\pm$ 0.10	94.51 $\pm$ 0.15	OOM
GraphGPS	88.94 $\pm$ 0.16	55.76 $\pm$ 0.23	OOM	95.06 $\pm$ 0.13	93.93 $\pm$ 0.12	OOM
NAGphormer	89.70 $\pm$ 0.19	71.51 $\pm$ 0.13	91.22 $\pm$ 0.14	95.49 $\pm$ 0.11	95.75 $\pm$ 0.09	97.34 $\pm$ 0.03

Comparison of Datasets

Table 2: Comparison of all models in terms of mean accuracy $\pm$ stdev (%) on large-scale datasets. The best results appear in **bold**.

Method	AMiner-CS	Reddit	Amazon2M
PPRGo	49.07 $\pm$ 0.19	90.38 $\pm$ 0.11	66.12 $\pm$ 0.59
GraphSAINT	51.86 $\pm$ 0.21	92.35 $\pm$ 0.08	75.21 $\pm$ 0.15
GRAND+	54.67 $\pm$ 0.25	92.81 $\pm$ 0.03	75.49 $\pm$ 0.11
NAGphormer	56.21 $\pm$ 0.42	93.58 $\pm$ 0.05	77.43 $\pm$ 0.24

Table 3: The accuracy (%) with or without structural encoding.

	Pubmed	CoraFull	CS	Computer	Photo	Physics	AMiner-CS	Reddit	Amazon2M
W/O-SE	89.06	70.42	95.52	90.44	95.02	97.10	55.64	93.47	76.98
With-SE	89.70	71.51	95.75	91.22	95.49	97.34	56.21	93.58	77.43
Gain	+0.64	+1.09	+0.23	+0.78	+0.47	+0.24	+0.57	+0.11	+0.45

Ablation Study

- SIN : Transformer layer에서 학습된 노드 자체의 표현만을 최종 출력으로 사용함
- SUM : 다른 hop의 이웃 정보를 합산하여 노드 표현을 만듦
- ATT : 0-hop 이웃(노드 자신)과 다른 k-hop 이웃 간의 어텐션 계수를 계산하여 중요도를 학습함

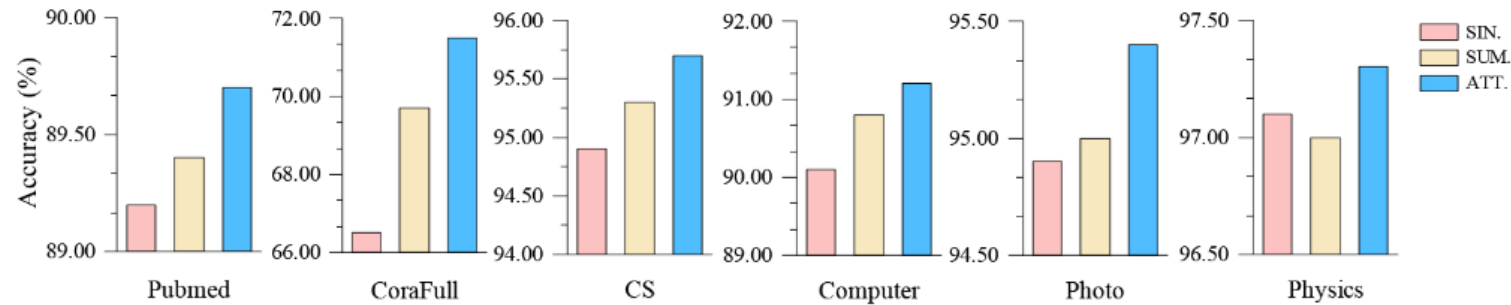
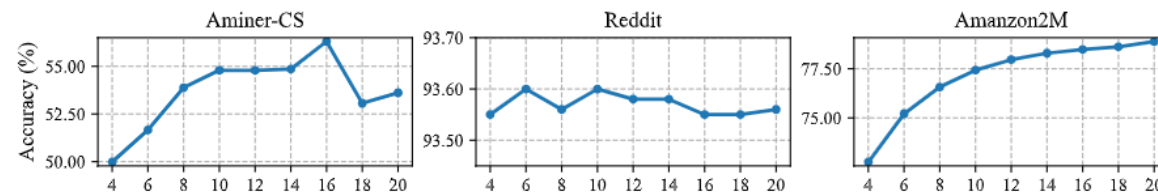


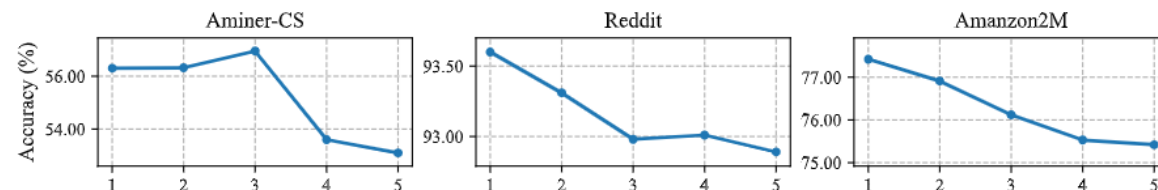
Figure 2: The performance of NAGphormer via different readout functions.

Parameter Study

- (a) Aminer-CS : 일정 수준까지는 더 많은 hop 의 이웃 정보를 활용하는 것이 성능 향상에 도움이 되지만, 과도하면 노이즈가 되거나 최적의 hop 수를 넘어서 성능이 저하됨
- (b) Aminer-CS : 적절한 깊이의 Transformer Layer는 성능을 향상시키지만, 너무 깊어지면 over-fitting 등의 문제로 성능이 저하될 수 있음
- (b) Reddit/Amazon2M : 깊은 Transformer 모델이 성능을 저하시킴



(a) On the number of propagation steps K



(b) On the number of Transformer layers L

Figure 3: Performance of NAGphormer on different parameters.

